

Decision-OS V13 Research Note v0.2

Post-Optimization Survival Architecture

Compound Loop Governance for AI Operations

Author: Shinichi Nagata

Version: v0.2 Research Note

Date: 2026-06-11 JST

Status: Canon Freeze / Prototype-Bound Draft

Working Rule

This v0.2 note intentionally keeps the main body minimal.

The purpose is not to include every V13 concept, but to define the readable entry point of V13:

- Compound Loop
- Capability without controllability
- Post-Optimization Survival Architecture
- Non-Dominating Pre-Positioning
- Re-entry Capacity
- Operator Boundary
- GO / HOLD / CAP / BLOCK

Related concepts such as Cost Memory, Branch Debt, Model Homeostasis, Failure Budget, Threshold Discovery Loop, Regime Shift Survival, and 0.99 Loop Diagnosis are treated as addendum candidates or prototype-derived extensions.

Abstract

This research note introduces Decision-OS V13 as a post-optimization survival architecture for AI-assisted operations.

As AI loops become easier to run, the central problem is no longer whether a task can be repeated, accelerated, or optimized. The central problem becomes which loops should be scaled, which should be capped, and which should be refused before they damage the Carrier that must continue.

V13 defines a Compound Loop as a loop that improves the starting condition of the next loop. A loop is not valuable merely because it repeats action, produces output, or completes a task. It is valuable only when it leaves transferable residue, reduces future cost, preserves re-entry capacity, and improves the next iteration without breaking Aspire or the Carrier.

The core claim is simple:

Capability without controllability is not intelligence.

V13 therefore shifts AI operation from output acceleration to loop governance. It does not reject optimization. Instead, it defines the survival form required after optimization succeeds too well.

The operational outcome of V13 is a minimal loop gate: **GO / HOLD / CAP / BLOCK**. A loop may proceed when it is positive-EV, residue-producing, controllable, and Carrier-preserving. It should be held when the sign or cost is unclear, capped when exposure must be bounded, and blocked when it damages Aspire, the Carrier, or future re-entry capacity.

This note intentionally keeps the main body minimal. Related concepts such as Cost Memory, Branch Debt, Model Homeostasis, Failure Budget, Threshold Discovery Loop, and 0.99 Loop Diagnosis are treated as addendum candidates or prototype-derived extensions.

In one line:

V13 is the discipline of turning 1.00 into 1.01 without destroying the Carrier that must keep compounding.

1. Introduction: When Loops Become Too Easy

AI-assisted operation is increasingly described through loops.

A system can plan, execute, observe, revise, and repeat. An agent can generate code, test it, repair errors, summarize results, and continue. A writing system can draft, review, rewrite, compress, and publish. A marketing system can post, observe, adjust, and post again. A trading or automation system can monitor, act, evaluate, and rerun.

This is useful. But it is not sufficient.

A loop can complete tasks while preserving the same fragile assumptions.

A loop can produce better outputs while increasing dependency.

A loop can reduce visible friction while increasing hidden cost.

A loop can accelerate work while increasing branch debt, exposure, or ruin risk.

A loop can make the present look productive while making the next loop harder.

The problem is not that AI cannot loop.

The problem is that easy loops can scale non-compounding behavior.

When loops are difficult to run, the main bottleneck is execution. The operator asks whether the task can be completed at all. But when AI makes loops cheap, fast, and repeatable, the bottleneck moves. The central question becomes which loops should be repeated, which loops should be capped, and which loops should be refused.

V13 begins from this shift.

The question is not only:

Can this loop run again?

The better question is:

Should this loop run again, and what should it improve before the next run?

A loop that repeats action without improving the next starting condition is only a replay loop. It may be useful for local completion, but it does not compound. A loop becomes valuable only when it leaves behind transferable residue: a better rule, a lower cost, a clearer threshold, a safer gate, a reusable template, a stronger evidence path, or a better re-entry condition.

In this sense, V13 does not treat loop repetition as intelligence. Repetition becomes intelligence only when it remains controllable, bounded, residue-producing, and Carrier-preserving.

This is why V13 fixes the following Canon:

Capability without controllability is not intelligence.

The same principle applies to AI operations, agentic workflows, code generation, research production, marketing experiments, trading loops, and automated business processes. A loop that expands faster than the operator

can understand, cap, stop, restart, or learn from is not a compound loop. It is uncontrolled repetition.

V13 therefore defines a post-optimization survival problem.

Optimization asks how to improve performance.

V13 asks what survives after performance becomes easy to improve.

If optimization succeeds too well, loops become abundant. When loops become abundant, the operator must govern them. This requires a minimal architecture for deciding whether the next loop should be run, held, capped, or blocked.

The purpose of this note is to define that architecture in its smallest usable form.

2. Compound Loop Definition

V13 defines a Compound Loop as a loop that improves the condition from which the next loop begins.

This definition separates Compound Loop from ordinary repetition.

A task may be repeated many times without compounding. An agent may execute, check, revise, and repeat while leaving the next iteration with the same unclear prompt, the same expensive model requirement, the same missing evidence, the same fragile handoff, or the same operator burden.

Such a loop may complete work, but it does not improve the operating condition of the next loop.

V13 therefore distinguishes four loop types.

2.1 Replay Loop

A Replay Loop repeats the task inside the same operating condition.

It asks:

Did the system try again?

A Replay Loop may be useful when the task is simple, reversible, and low-cost. However, it does not necessarily improve the next iteration. It may consume time, tokens, attention, or money while preserving the same structural weakness.

Replay is not failure. But replay is not compounding.

2.2 Correction Loop

A Correction Loop fixes errors inside a defined task.

It asks:

Did the system repair the output?

Correction Loops are useful for debugging, editing, formatting, quality control, and ordinary task completion. They improve the current artifact, but they may still leave the next loop unchanged.

A corrected output is not automatically a compound loop.

2.3 Optimization Loop

An Optimization Loop improves performance toward a defined objective.

It asks:

Did the system perform better against the target?

Optimization Loops can improve speed, quality, accuracy, conversion, or efficiency. However, optimization can still become dangerous when the target is too narrow, when cost is hidden, when the operator cannot stop the process, or when improvement in one metric damages the Carrier.

Optimization is valuable only when the loop remains controllable.

2.4 Compound Loop

A Compound Loop improves the future operating condition.

It asks:

What changed that makes the next loop cheaper, safer, clearer, more reusable, or more restartable?

A loop qualifies as compound only when it leaves a transferable residue that improves future iteration.

Examples of transferable residue include:

- a reusable template,
- a better prompt,
- a reduced model requirement,
- a clearer threshold,
- a stronger stop condition,

- a better test,
- a cleaner handoff,
- a reusable evidence path,
- a smaller next action,
- a safer cap,
- a clearer operator boundary.

The minimal form is:

Compound Loop = positive-EV detection + variable selection + ruin-risk cap + transferable residue + Carrier preservation.

The key variable is not repetition count.

The key variable is whether the next loop begins from a better state.

A loop becomes 1.01 when it improves at least one future condition without damaging the Carrier that must continue. A loop remains 1.00 when it merely reproduces the same operating condition. A loop becomes 0.99 when it consumes resources, increases dependency, narrows re-entry, or makes future operation harder.

The purpose of V13 is not to maximize loop count. It is to select the variable that makes the next loop worth running.

In one line:

A Compound Loop improves the condition from which the next loop begins.

3. Post-Optimization Survival Architecture

V13 is not an alternative to optimization.

It is the survival form required after optimization succeeds too well.

Optimization asks how to improve performance against a target. This is necessary. A system that cannot improve speed, quality, accuracy, cost, or execution reliability will fail in many practical environments.

However, optimization changes the problem once it becomes effective.

When optimization is weak, the operator asks:

Can this process improve?

When optimization becomes strong, the operator must ask:

Should this improvement be scaled?

AI-assisted systems increasingly make optimization cheap. A model can generate more variants, test more alternatives, rewrite more drafts, repair more code, and produce more plans. This creates a new failure mode: the system can optimize loops that should not be expanded.

A loop may become faster while becoming less governable.

A loop may become cheaper locally while increasing hidden review cost.

A loop may improve output quality while increasing dependency on premium models.

A loop may increase production while reducing operator recovery capacity.

A loop may increase options while creating branch debt.

A loop may increase confidence while reducing re-entry capacity.

In this environment, survival requires more than performance improvement.

It requires loop governance.

V13 treats AI operation as a post-optimization survival problem. The question is not whether the system can run a loop. The question is whether the loop should be scaled, held, capped, or blocked.

This shifts the operating center from acceleration to survivability.

Optimization tends to focus on the current objective. V13 focuses on the loop's effect on the next loop, the operator, and the Carrier. A loop that improves a metric while damaging future continuation is not a compound loop. It is pre-collapse acceleration.

The same distinction applies to research, coding, automation, marketing, trading, and business operations. A workflow may appear successful because it produces visible output. But if each loop increases hidden cost, reduces restartability, damages trust, narrows branching, or consumes recovery capacity, then the system is not compounding. It is converting optimization into depletion.

V13 therefore introduces a survival test after optimization:

What remains after the optimized loop runs?

If the answer is only output, the loop may be useful but not compound.

If the answer includes transferable residue, lower future cost, clearer boundaries, preserved re-entry capacity, and a safer next loop, then the loop

begins to compound.

This is why V13 does not promise victory.

No loop governance architecture can guarantee that an operator will win. External conditions may change. Markets may shift. Users may not respond. Competitors may enter. A technically correct loop may still fail.

V13 makes a narrower and stronger promise:

No promise of victory.

A stronger promise of non-collapse.

The goal is to prevent the operator from scaling loops that destroy the Carrier required for future action.

In this sense, V13 belongs after V10, V11, and V12 in the Decision-OS sequence.

V10 protects the trajectory from fixed-goal collapse.

V11 protects long-horizon operation from memory overload and unreconnectable forgetting.

V12 protects completion from false closure and restart failure.

V13 protects iteration from non-compounding repetition.

Together, these layers move AI operation away from one-shot output quality and toward survivable long-horizon operation.

V13's role is specific:

When loops become easy, govern what gets repeated.

In one line:

V13 is not an alternative to optimization. It is the survival form required after optimization succeeds too well.

4. Controllability Canon

V13 fixes the following Canon:

Capability without controllability is not intelligence.

This statement does not deny capability. Capability matters. A system that cannot produce, reason, code, write, evaluate, or execute has little operational value.

However, capability alone is incomplete.

A capable system can still be dangerous, wasteful, dependency-producing, or Carrier-damaging if the operator cannot govern the loop it creates.

In V13, a loop is controllable only when the operator can still:

- understand why the loop is running,
- identify what variable is being improved,
- cap cost, exposure, authority, and iteration count,
- stop the loop before irreversible damage,
- preserve or extract transferable residue,
- downshift the model or process when stable,
- escalate only when uncertainty requires it,
- preserve re-entry capacity,
- and keep the final Seat with the operator.

A loop that lacks these properties may still produce impressive output. But impressive output is not the same as operational intelligence.

A code agent may generate a large feature quickly, but if the next agent cannot understand, test, rollback, or continue it, the loop has not compounded.

A writing agent may produce a polished essay, but if the reasoning path disappears, the output becomes dependency.

A business automation may create leads, posts, or messages, but if it scales a negative-EV process, it turns automation into loss.

A trading loop may execute faster, but if size, risk, and re-entry are not capped, speed becomes ruin acceleration.

V13 therefore evaluates capability through controllability.

The question is not only:

What can this system do?

The V13 question is:

Can this system's loop be governed across repeated operation?

Controllability is not a decorative safety feature. It is part of intelligence under long-horizon operation. If an operator cannot understand, cap, stop, restart, or learn from a loop, then the loop is not fully intelligent in the operational sense.

This is why V13 distinguishes output quality from operational survivability.

Output quality can be demonstrated in minutes.

Operational survivability can only be proven across loops.

Controllability also prevents a common AI-era illusion: the belief that generation capacity equals operator capacity.

AI can generate more paths than the operator can carry. It can produce plans faster than the Carrier can absorb them. It can make expansion look available before the operator has cost memory, trust, evidence, recovery capacity, or re-entry structure.

V13 rejects this substitution.

Generation capacity belongs to the AI system.

Carrying capacity belongs to the operator and Carrier.

A loop is valid only when these two capacities remain aligned.

This is especially important when premium models are used. A high-capability model may produce excellent outputs, but if every future loop requires the same expensive model, heavy context, and human review burden, the system may become more dependent rather than more compound.

Therefore, V13 asks whether premium capability becomes lower-layer structure.

If premium output becomes a reusable rule, template, test, threshold, or route, it compounds.

If premium output remains necessary every time, it becomes dependency.

The purpose of controllability is not to slow down intelligence. It is to make intelligence reusable, bounded, and survivable.

In one line:

Capability without controllability is not intelligence.

5. Non-Dominating Pre-Positioning

V13 introduces Non-Dominating Pre-Positioning as a future-line strategy.

A Future Line is not a bet on hype. It is a low-cost pre-positioning loop that becomes valuable when external signals raise the slope.

This distinction is important.

Many operators misunderstand early entry as domination. They try to enter a future line with too much capital, too much public exposure, too much commitment, or too much identity attachment before the external signal has matured. This can damage the Carrier before the line becomes legible.

V13 rejects this pattern.

The goal is not to dominate an uncertain future line.

The goal is to enter cheaply enough that waiting, learning, and preserving optionality remain possible.

Non-Dominating Pre-Positioning means entering an undervalued future line cheaply enough to accumulate time-interest and operating residue, without distorting the market, overexposing the operator, or breaking the Carrier.

The key advantage is not merely early entry.

The advantage is being able to operate cheaply when others enter expensively.

Early entry alone can be fragile. If the operator enters too heavily, the position may become a burden. If the operator must defend the position, explain it publicly, keep spending, or maintain attention before the signal matures, then the future line may damage re-entry capacity.

A valid future-line loop must therefore satisfy five conditions:

1. External Signal

There must be some external signal: market movement, regulation, funding, terminology, user pain, buyer behavior, technical shift, or institutional attention.

2. Internal Residue

The operator must already possess some residue: prior learning, failed attempts, templates, language, evidence, infrastructure, relationships, or AI-loop structure.

3. Cost Advantage

The operator must be able to begin more cheaply than later entrants.

4. Carrier Fit

The loop must fit the operator's current Carrier, including time, money, attention, credibility, health, and recovery capacity.

5. Reversible Expansion

The loop must be expandable under a cap. If the signal fails, the operator must be able to close or downshift without losing re-entry capacity.

A Future Line should be promoted only when external signals meet internal residue.

Without external signal, the loop is only speculation.

Without internal residue, the loop is only reaction.

Without cost advantage, the operator enters late and expensively.

Without Carrier fit, the line may grow while damaging the operator.

Without reversible expansion, the loop becomes a trap.

This is why V13 treats future-line entry as a capped loop, not as a domination attempt.

The purpose of Non-Dominating Pre-Positioning is to accumulate enough structure before the public slope rises, while preserving the ability to stop, wait, or re-enter.

This principle also prevents branch inflation. AI can generate many plausible future lines, but each branch creates future obligation: monitoring, maintenance, updating, explanation, closure, and re-entry management. Opening many future lines without caps does not create optionality. It creates branch debt.

Therefore, V13 favors small, bounded, residue-producing entries.

A good future-line loop does not ask:

How much can we capture now?

It asks:

Can we enter cheaply, learn safely, preserve the Carrier, and be ready if the external signal strengthens?

This is the discipline of non-domination.

The real strength of a Compound Loop is not the range it can dominate, but the discipline of what it refuses to pursue.

In one line:

The advantage is not entering early. The advantage is being able to operate cheaply when others enter expensively.

6. Re-entry Capacity and Operator Boundary

V13 protects Re-entry Capacity.

Re-entry Capacity is the ability to return later with enough money, time, attention, credibility, health, trust, monitoring ability, and decision capacity to act when conditions improve.

This concept changes how loops, branches, and positions are evaluated.

A branch is not valuable if it destroys the ability to re-enter.

A position is not safe if holding it prevents future action.

A loop is not compounding if it consumes the Carrier needed for the next loop.

V13 therefore does not protect branches for their own sake.

It protects the capacity to re-enter.

Re-entry Capacity includes:

- capital remaining,
- time remaining,
- attention remaining,
- credibility remaining,
- physical and cognitive recovery remaining,
- self-trust remaining,
- AI-trust remaining,
- monitoring ability remaining,
- rollback or exit ability remaining,
- the ability to respond when a future trigger appears.

This is especially important in AI-assisted operation because AI makes it easy to open new branches. A model can generate more product ideas, more markets, more posts, more repositories, more automations, more investment theses, and more future lines. Each branch may look small at the moment of creation.

But every branch creates a future obligation.

It must be monitored.

It must be maintained.

It must be updated.

It must be closed.

It must be explained.

It must be re-entered or abandoned under some condition.

If the operator opens too many branches without closure conditions, optionality becomes burden. What appeared to be expansion becomes branch debt.

V13 therefore treats closure as a positive operation.

Closure is not the loss of a branch.

Closure is the recovery of re-entry capacity.

A branch may be closed when it no longer preserves or increases future optionality. Closing a branch can return attention, capital, recovery capacity, and monitoring bandwidth to the operator. This makes future entry possible.

In this sense, cash is not inactivity when it preserves re-entry capacity for a better time window. Waiting is not failure when the waiting state preserves the ability to respond to a stronger signal. A closed loop is not necessarily a dead loop if it leaves residue, evidence, and a re-entry trigger.

V13 evaluates every loop through the following question:

After this loop, can the operator still re-enter?

If the answer is no, the loop is dangerous even when the local upside is attractive.

This leads to the Operator Boundary.

A loop is not high-EV in the abstract. It is high-EV only when it fits the operator's Aspire, Carrier, capital, recovery structure, trust, skill, social context, and current phase.

The same loop can be positive for one operator and destructive for another.

A future line may be promising in the market but wrong for the operator.

A business model may be profitable for a team but destructive for a solo researcher.

A trading strategy may be positive for a well-capitalized desk but ruinous for a small account.

A coding agent workflow may be efficient for an engineer but confusing for a non-engineer if the handoff and rollback path are weak.

A public posting loop may build distribution for one person while consuming the credibility or attention of another.

Therefore, V13 rejects abstract loop maximalism.

The operator boundary asks:

Can this operator run this loop without breaking the Carrier that must continue?

If the answer is no, the loop should not scale, even if the opportunity looks attractive.

This prevents a common AI-era failure:

Because AI can generate the path, the operator assumes they can carry the path.

V13 separates generation capacity from operator capacity.

AI may expand the map.

The operator must still survive the terrain.

A valid loop must preserve both Aspire and the Carrier. Aspire gives the loop direction. The Carrier gives the loop continuity. If the Carrier is damaged, the loop may produce a short-term output but fail as a long-horizon operation.

The V13 rule is simple:

Do not protect the branch. Protect the capacity to re-enter.

And:

A line that grows the market but damages the operator is not a Future Line for that operator.

In one line:

Do not protect the branch. Protect the capacity to re-enter.

7. GO / HOLD / CAP / BLOCK Gate

V13 uses a four-outcome loop gate:

GO / HOLD / CAP / BLOCK

The purpose of this gate is to decide whether a loop should be repeated, observed, bounded, or refused.

This gate is not a moral judgment. It is an operational control surface for AI-assisted loops.

A loop may be promising but unclear.

A loop may be useful but too expensive.

A loop may be positive under small exposure but dangerous when scaled.

A loop may produce output while damaging the Carrier.

A loop may be valid for one more iteration but invalid as an automated process.

Therefore, V13 does not reduce loop governance to GO or BLOCK.

Many loops are not wrong.

They are only valid under a cap.

7.1 GO

GO means the loop may proceed.

A loop receives GO when the following conditions are satisfied:

- the loop appears positive-EV,
- the loop is controllable,
- ruin risk is bounded,
- the Carrier is preserved,
- Aspire is not damaged,
- transferable residue is likely,
- re-entry capacity remains available,
- and the next loop is expected to begin from a better condition.

GO does not mean the outcome is guaranteed. It means the loop is worth running under the current conditions.

A GO loop should still leave a record of what variable it intends to improve.

7.2 HOLD

HOLD means the loop should not be scaled yet.

A loop receives HOLD when the sign, cost, residue, Carrier impact, or re-entry condition is unclear.

HOLD is not indecision. It is controlled non-commitment.

The purpose of HOLD is to prevent premature scaling before the operator understands whether the loop is 1.01, 1.00, or 0.99.

HOLD should define what must be observed before the next decision. A HOLD without a recheck condition becomes drift. A valid HOLD includes:

- what is uncertain,
- what will be observed,
- when the loop will be rechecked,
- what would convert HOLD into GO,
- what would convert HOLD into CAP,
- and what would convert HOLD into BLOCK.

7.3 CAP

CAP means the loop may proceed only under a fixed limit.

This is the central practical outcome of V13.

Many loops are not safe at full scale but are useful under bounded exposure. A CAP allows the operator to learn without converting uncertainty into irreversible commitment.

A CAP may limit:

- money,
- time,
- token cost,
- model cost,
- human review cost,
- public exposure,
- automation authority,
- iteration count,
- number of branches,
- trading size,
- publishing scope,
- or dependency on premium models.

CAP is the operational form of bounded learning.

It permits small failure early while preventing catastrophic failure later. It also allows the operator to discover thresholds without pretending that the first threshold is correct.

A CAP loop should define:

1. the allowed exposure,
2. the stop condition,
3. the recheck condition,
4. the expected residue,
5. the rollback or closure path,
6. and the maximum Carrier impact.

Without these, CAP becomes vague permission.

A valid CAP says:

Run the loop, but only within this boundary.

7.4 BLOCK

BLOCK means the loop must not proceed under current conditions.

A loop receives BLOCK when it is negative-EV, uncontrollable, Carrier-damaging, Aspire-damaging, ruin-risk exposed, or dependent on irreversible success assumptions.

BLOCK does not mean the line is permanently dead. It means the current form of the loop is not admissible.

A blocked loop may later become eligible if its structure changes: lower exposure, clearer evidence, better residue path, stronger Carrier fit, or a safer re-entry condition.

However, until such a change occurs, the loop should not run.

BLOCK is required when:

- the loop damages the Carrier,
- the loop reduces re-entry capacity,
- the loop cannot be stopped,
- the loop lacks rollback,

- the loop scales a known defect,
- the loop converts uncertainty into irreversible commitment,
- the loop requires success to avoid ruin,
- or the loop shifts the Seat away from the operator.

7.5 Minimal Loop Record

For practical use, the V13 gate can be recorded in a minimal loop record:

V13 Loop Record

1. Previous Loop:

What was the prior loop?

2. Residue:

What transferable residue was produced?

3. Next Variable:

Which variable should improve the next loop?

4. Carrier Impact:

What did the loop cost the operator or Carrier?

5. Re-entry Capacity:

Can the operator still re-enter later?

6. Gate:

GO / HOLD / CAP / BLOCK

7. Cap or Recheck:

If CAP, what is the limit?

If HOLD, what is the recheck condition?

If BLOCK, what must change before reconsideration?

8. Next Loop Command:

What is the smallest valid next loop?

This minimal record connects V13 to implementation.

In the Decision-OS sequence, V12 asks whether a task is truly complete and restartable. V13 asks whether the next loop should be run, held, capped, or blocked.

The connection is simple:

V12 prevents false completion.

V13 prevents non-compounding repetition.

A V12 completion record can therefore become the input to a V13 loop record. The output is not merely "continue." The output is a governed next loop.

In one line:

Many loops are not wrong. They are only valid under a cap.

8. Boundary to Prototype

This v0.2 note is intentionally limited.

It does not attempt to include every concept generated during V13 development. It does not define a complete product. It does not claim empirical validation. It does not present a full implementation of loop governance.

Its purpose is narrower:

This note freezes the Canon; the prototype will test the operating surface.

The Canon fixed here consists of the following minimal components:

1. Compound Loop

A loop is valuable only when it improves the condition from which the next loop begins.

2. Post-Optimization Survival Architecture

V13 is the survival form required after optimization succeeds too well.

3. Controllability Canon

Capability without controllability is not intelligence.

4. Non-Dominating Pre-Positioning

The advantage is not entering early, but operating cheaply when others enter expensively.

5. Re-entry Capacity

Do not protect the branch. Protect the capacity to re-enter.

6. Operator Boundary

A loop is high-EV only when it fits the operator's Aspire, Carrier, resources, and context.

7. GO / HOLD / CAP / BLOCK

Loops should be governed by proceed, observe, bound, or refuse decisions.

Related concepts are intentionally deferred.

Cost Memory, Branch Debt, Model Homeostasis, Failure Budget, Threshold Discovery Loop, Regime Shift Survival, Formatting Dependency, 0.99 Loop Diagnosis, and Future-Line Trigger Rules are all important. However, placing all of them in the main body would make the entry point too heavy.

They should be treated as addendum candidates or prototype-derived extensions.

The immediate prototype path is the V12 → V13 LoopKit.

In the Decision-OS sequence, V12 asks:

Is this work actually complete in a way that the future self or next agent can restart?

V13 asks:

Given that completion state, should the next loop be run, held, capped, or blocked?

This creates a practical handoff:

```
V12 Completion Record
  ↓
V13 Loop Record
  ↓
GO / HOLD / CAP / BLOCK
  ↓
Next Loop Command
```

The prototype should therefore begin as a minimal loop governance kit attached to the end of existing AI work.

It does not need to be a large application at first. The smallest useful version is a structured record that reads the previous completion state and outputs:

- transferable residue,
- next variable to improve,
- Carrier impact,
- re-entry capacity,
- loop gate,
- cap or recheck condition,

- and the smallest valid next loop.

This prototype will reveal what the paper alone cannot determine.

For example:

- whether users understand CAP,
- whether HOLD feels useful or vague,
- whether Cost Memory must return to the main body,
- whether Operator Boundary can be captured in a short form,
- whether Model Homeostasis should be automated or left as guidance,
- whether Re-entry Capacity can be evaluated without becoming too heavy,
- and whether the V12 → V13 connection is natural in real workflows.

For this reason, v0.2 should not pretend to be the final V13 paper.

The correct sequence is:

```
v0.1 = discovery of Compound Loop
v0.2 = Canon Freeze
prototype = operating-surface discovery
v1.0 = paper after prototype feedback
```

This preserves speed without hiding incompleteness.

V13 v0.2 defines the theory enough to build from it.

The prototype tests what the theory becomes when it touches real loops.

In one line:

This note freezes the Canon; the prototype will test the operating surface.

9. Conclusion

AI loops are becoming easy to run.

This changes the central problem of operation.

When loops are difficult, execution is the bottleneck. When loops become easy, governance becomes the bottleneck. The operator must decide which loops should be repeated, which should be held, which should be capped, and which should be refused.

V13 defines Compound Loop Governance for this condition.

A loop is not valuable merely because it repeats action.

A loop is not valuable merely because it produces output.

A loop is not valuable merely because it improves a local metric.

A loop becomes valuable when it improves the condition from which the next loop begins.

This note fixed the minimal Canon of V13:

- A Compound Loop improves the next starting condition.
- V13 is a post-optimization survival architecture.
- Capability without controllability is not intelligence.
- Future-line advantage comes from low-cost, non-dominating pre-positioning.
- Re-entry Capacity is more important than protecting any single branch.
- Loop EV must be bounded by the operator's Aspire, Carrier, resources, and context.
- Loops should be governed through GO / HOLD / CAP / BLOCK.

The purpose of V13 is not to reject optimization, acceleration, automation, or AI loops.

The purpose is to prevent loops from scaling faster than they can be governed.

A loop that cannot be understood, capped, stopped, restarted, or learned from is not operationally intelligent, no matter how impressive its output appears. A loop that produces output but no transferable residue is not compounding. A loop that improves performance while damaging the Carrier is not evolution. It is pre-collapse acceleration.

V13 therefore moves the focus from output to operating condition.

The question is not:

What did this loop produce?

The better question is:

What did this loop leave behind that makes the next loop cheaper, safer, clearer, more restartable, or more controllable?

If the answer is clear, the loop may receive GO.

If the answer is uncertain, the loop should HOLD.

If the loop is useful only within limits, it should receive CAP.

If the loop damages Aspire, the Carrier, or re-entry capacity, it should receive BLOCK.

This is the minimum architecture required when AI-assisted loops become abundant.

V13 does not promise victory.

Markets may shift. Users may not respond. Competitors may enter. Timing may fail. External conditions may dominate. A well-governed loop can still lose.

But V13 makes a stronger and narrower promise:

No promise of victory.

A stronger promise of non-collapse.

The next step is not to add every V13 concept into the paper. The next step is to build the V12 → V13 LoopKit and test the operating surface.

V12 prevents false completion.

V13 prevents non-compounding repetition.

Together, they create the smallest practical bridge from completed work to governed iteration.

In one line:

V13 is the discipline of turning 1.00 into 1.01 without destroying the Carrier that must keep compounding.